

Artificial Intelligence Specialization: All Questions

AI Specialization 1: SMU

Learnability models: PAC and online. Learnability of conjunctions and disjunctions. Multi-armed bandit problem. Reinforcement learning.

Sources used: official CTU CourseWare page for Symbolic Machine Learning, which lists reinforcement learning and computational learning theory as course parts; CTU state-exam topic list; standard learning theory and RL material.

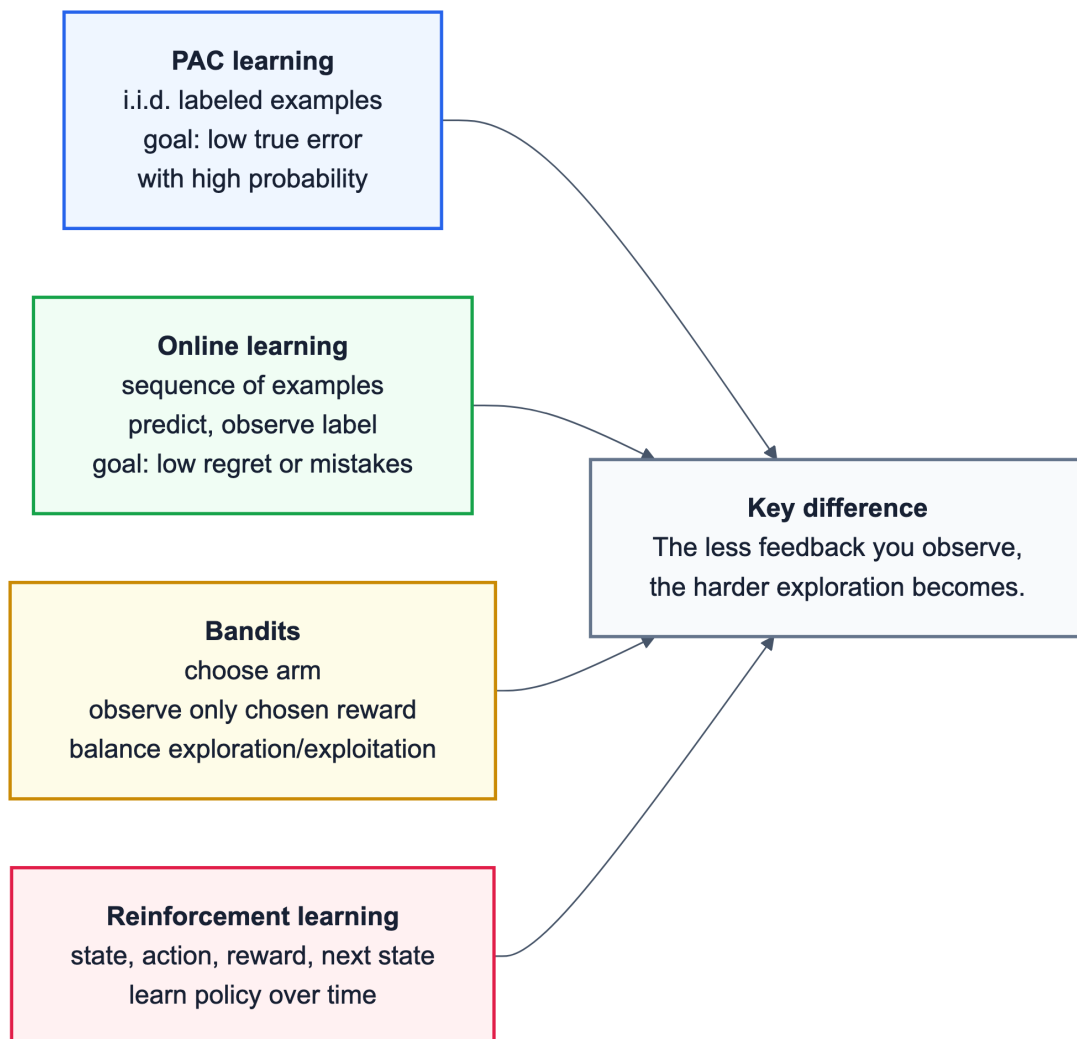


Figure 1: Learning-feedback settings: PAC, online learning, bandits, and reinforcement learning differ mainly in what feedback is observed.

SMU in Plain English

SMU is about learning from feedback. The main question is always: what feedback do we see, what is the learner trying to minimize, and how much exploration is needed?

1. Identify feedback.

Full label, mistake, chosen reward, or delayed reward?

2. Name the goal.

Low error, few mistakes, low regret, high return.

3. Give the update.

Eliminate, explore, estimate values, or improve policy.

PAC learning

Batch examples from a fixed distribution.

- Goal: low true error with high probability.
- Needs sample complexity in epsilon and delta.
- VC dimension controls learnability.

Online learning

Predict one example at a time.

- Goal: few mistakes or low regret.
- Can be adversarial sequence.
- Conjunction/disjunction learners eliminate inconsistent literals.

Bandits

You only observe reward for the arm you pulled.

- Explore/exploit tension is central.
- Regret compares against best fixed arm.
- UCB = average reward plus uncertainty bonus.

Reinforcement learning

Actions change future states and rewards.

- Value iteration assumes known model.
- TD/Q-learning learn from sampled transitions.
- Policy search optimizes policy directly.

Key contrast

Less feedback makes learning harder.

- PAC: label for sampled example.
- Bandit: only chosen reward.
- RL: chosen reward plus next state, delayed consequences.

How to answer

Always say the feedback model first.

- What is observed?
- What is optimized?
- What algorithm handles exploration?

Exam sentence: In bandits the learner does not see rewards of unchosen arms, so algorithms like UCB add an uncertainty bonus to try promising but under-sampled actions.

Three traps

- PAC assumes i.i.d. data; online learning may be adversarial.
- Bandit regret is not the same as supervised classification error.
- Q-learning estimates action values, not just state values.

Use this page to keep the feedback settings separate.

1. PAC and Online Learnability

PAC model

PAC means **P**robably **A**pproximately **C**orrect. We have:

- instance space X ,
- labels $Y = \{0, 1\}$, or $\{-1, +1\}$,
- unknown distribution D over X ,
- unknown target concept $c \in C$,
- training examples $(x_i, c(x_i))$ drawn i.i.d. from D .

A learner outputs hypothesis $h \in H$. Its true error is

$$err_D(h) = \Pr_{x \sim D}[h(x) \neq c(x)].$$

A concept class C is PAC-learnable by H if for every $\epsilon, \delta \in (0, 1)$ and every distribution D , after

$$m = poly(1/\epsilon, 1/\delta, n, size(c))$$

i.i.d. examples, the learner outputs h such that

$$\Pr[err_D(h) \leq \epsilon] \geq 1 - \delta.$$

It is **efficiently PAC-learnable** if the learner also runs in polynomial time.

Online model

In online learning, examples arrive sequentially. At round t :

1. learner receives x_t ,
2. predicts $\hat{y}_t$,
3. true y_t is revealed,
4. learner updates its state.

No i.i.d. assumption is needed; the sequence may be adversarial. Performance is measured by mistakes or regret.

Mistake-bound model: for realizable sequences labeled by some $c \in C$, learner should make at most $M(n)$ mistakes.

Regret model: compare cumulative loss to best fixed hypothesis in hindsight:

$$R_T = \sum_{t=1}^T \ell(\hat{y}_t, y_t) - \min_{h \in H} \sum_{t=1}^T \ell(h(x_t), y_t).$$

Sublinear regret $R_T = o(T)$ means average performance converges to the best comparator.

PAC vs. online

Main differences:

Aspect	PAC	Online
Data	i.i.d. samples from fixed distribution	sequential, possibly adversarial
Goal	low true error with high probability	few mistakes or sublinear regret
Feedback	training labels before final hypothesis	label/loss after each prediction
Output	final predictor	sequence of predictions

Relations:

- A finite mistake-bound online learner implies PAC learnability in the realizable case: simulate online learning on i.i.d. samples and use the final hypothesis.
- PAC learnability does not automatically imply low-mistake online learnability against adversarial sequences.
- Finite VC dimension characterizes distribution-free PAC learnability for binary classification.
- Finite Littlestone dimension characterizes online learnability in the realizable adversarial setting.

VC dimension

Set $S = \{x_1, \dots, x_m\}$ is **shattered** by H if for every labeling of S , there exists $h \in H$ realizing it.

The VC dimension $VC(H)$ is the size of the largest shattered set, or ∞ if arbitrarily large sets can be shattered.

For finite VC dimension d , a typical realizable PAC sample bound is

$$m = O\left(\frac{d \log(1/\epsilon) + \log(1/\delta)}{\epsilon}\right).$$

For agnostic learning, a typical bound is

$$m = O\left(\frac{d + \log(1/\delta)}{\epsilon^2}\right).$$

Necessary and sufficient condition: a binary class is distribution-free PAC-learnable iff it has finite VC dimension.

Simple explanation

VC dimension measures how many points a model class can label in every possible way. If the class can shatter too many points, it can memorize noise. Finite VC dimension means enough random examples will eventually control overfitting.

2. Learnability of Conjunctions and Disjunctions

Let instances be Boolean vectors $x \in \{0, 1\}^n$. Literals are variables x_i and negations $\neg x_i$.

Conjunctions in the PAC model

Class: conjunctions of literals, e.g.

$$c(x) = x_1 \wedge \neg x_3 \wedge x_5.$$

Efficient PAC algorithm for realizable data:

1. Start with all $2n$ literals in the hypothesis: $h = x_1 \wedge \neg x_1 \wedge \dots \wedge x_n \wedge \neg x_n$.
2. For every positive example x : remove every literal falsified by x .
3. Ignore negative examples.
4. Output remaining conjunction.

Reason: every literal falsified by a positive example cannot be in the target. Every remaining literal is consistent with all positive examples. If the sample is large enough, any harmful extra literal would probably be exposed by a positive example.

Efficient: $O(mn)$ time.

Example

With variables x_1, x_2 , start with $x_1 \wedge \neg x_1 \wedge x_2 \wedge \neg x_2$. Positive example $(x_1 = 1, x_2 = 0)$ removes $\neg x_1$ and x_2 . Positive example $(1, 1)$ removes $\neg x_2$. The remaining hypothesis is x_1 .

Disjunctions in the PAC model

Disjunctions can be learned directly by the dual algorithm:

1. Start with all literals in a disjunction.
2. For every negative example, remove literals made true by that example.
3. Ignore positive examples.

Or reduce disjunction learning to conjunction learning using De Morgan:

$$l_1 \vee \dots \vee l_k = \neg(\neg l_1 \wedge \dots \wedge \neg l_k).$$

Online learning of conjunctions

Use the same elimination idea:

1. Start with all literals.
2. Predict using current conjunction.
3. If a positive example is misclassified as negative, remove all literals falsified by it.
4. Negative mistakes do not require changing the hypothesis in the realizable monotone elimination version, but in the full literal class consistency must be maintained.

Each literal is removed at most once, so the number of updates is at most $2n$. Thus conjunctions are efficiently online learnable with a finite mistake bound.

Simple explanation

The conjunction learner starts overly strict and relaxes itself when positive examples prove that some literal cannot be required. Every mistake teaches it to delete at least one wrong requirement, so it cannot keep making the same kind of mistake forever.

Online learning of disjunctions

Use the dual algorithm or De Morgan reduction. Again the mistake/update bound is $O(n)$ in the realizable setting.

Learning by reduction

Many simple Boolean classes reduce to conjunctions/disjunctions:

- Monotone conjunctions: use only positive literals.
- Monotone disjunctions: dual.
- k -CNF: conjunction of clauses; treat each possible clause of size at most k as a Boolean feature, then learn a conjunction. Efficient for fixed k because the number of clauses is $O(n^k)$.
- k -DNF: reduce dually to k -CNF by negation.

The key trick is to transform the representation into a feature space where the target becomes a conjunction or disjunction.

3. Multi-Armed Bandits

Problem definition

There are K arms. At round $t = 1, \dots, T$:

1. learner chooses arm $A_t \in \{1, \dots, K\}$,
2. receives reward R_t drawn from the selected arm distribution,
3. observes only that reward.

Let arm means be $\mu_i = \mathbb{E}[R|A = i]$ and $\mu^* = \max_i \mu_i$.

Regret

Expected regret:

$$\mathbb{E}[R_T] = T\mu^* - \mathbb{E} \left[\sum_{t=1}^T R_t \right].$$

Equivalently:

$$\mathbb{E}[R_T] = \sum_{i: \Delta_i > 0} \Delta_i \mathbb{E}[N_i(T)],$$

where $\Delta_i = \mu^* - \mu_i$ and $N_i(T)$ is the number of times arm i was pulled.

Goal: balance **exploration** of uncertain arms and **exploitation** of arms currently believed best.

Epsilon-greedy

At each round:

- with probability $1 - \epsilon$, choose arm with largest empirical mean,
- with probability ϵ , choose a random arm.

With fixed ϵ , regret is linear because exploration never vanishes. With decaying ϵ_t , regret can improve, but the schedule is problem-sensitive.

Limitations:

- explores uniformly, wasting pulls on obviously bad arms,
- sensitive to ϵ ,
- weaker theoretical guarantees than confidence-bound or posterior-sampling methods.

UCB

Upper Confidence Bound chooses the arm with largest optimistic estimate:

$$A_t = \arg \max_i \left(\hat{\mu}_i(t) + \sqrt{\frac{2 \ln t}{N_i(t)}} \right).$$

The second term is an uncertainty bonus. Arms with few samples get high bonus; arms with many samples rely mostly on empirical mean.

For bounded rewards, UCB1 has logarithmic regret:

$$\mathbb{E}[R_T] = O\left(\sum_{i:\Delta_i>0} \frac{\ln T}{\Delta_i}\right).$$

Simple explanation

UCB is “optimism under uncertainty.” If an arm has not been tried much, pretend it might be better than it currently looks. As evidence accumulates, the uncertainty bonus shrinks and the algorithm naturally focuses on the best arms.

Example

At time $t = 100$, suppose arm A has empirical mean 0.6 after 25 pulls, and arm B has empirical mean 0.55 after 4 pulls. UCB gives B a larger uncertainty bonus because it has been tried less. The algorithm may choose B even though its current average is lower.

Thompson sampling

Thompson sampling is Bayesian probability matching:

1. Maintain posterior distribution over each arm’s mean.
2. Sample θ_i from each posterior.
3. Pull arm $\arg \max_i \theta_i$.
4. Update posterior using observed reward.

Example for Bernoulli arms:

- prior $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$,
- after success: $\alpha_i \leftarrow \alpha_i + 1$,
- after failure: $\beta_i \leftarrow \beta_i + 1$.

Advantages:

- simple,
- naturally balances exploration/exploitation,
- strong empirical performance and logarithmic regret under standard assumptions.

4. Reinforcement Learning

Markov Decision Process

An MDP is

$$(S, A, P, R, \gamma),$$

where S are states, A actions, $P(s'|s, a)$ transition probabilities, $R(s, a, s')$ rewards, and $\gamma \in [0, 1)$ discount factor.

A policy $\pi(a|s)$ defines behavior. The state-value function is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s \right].$$

The action-value function is

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s, A_0 = a \right].$$

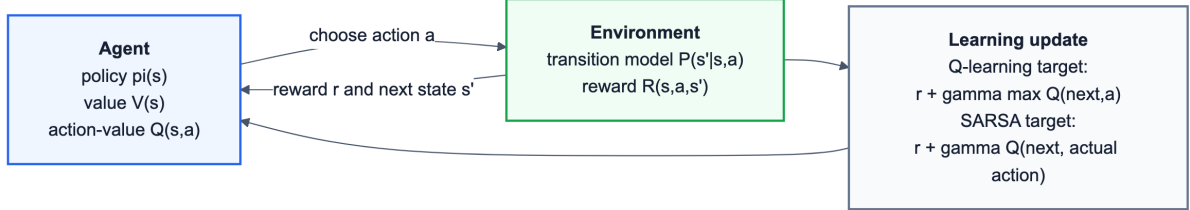


Figure 2: Reinforcement-learning loop: the agent chooses actions, the environment returns rewards and next states, and the value estimate is updated.

Optimal policy and Bellman equations

Optimal value:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')].$$

An optimal policy chooses

$$\pi^*(s) \in \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')].$$

Value iteration

For known model P, R :

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

Repeat until values change little. Then extract greedy policy. Converges for discounted finite MDPs.

Direct utility estimation

Estimate $V^\pi(s)$ from sampled returns:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

Average observed returns following visits to state s . This is Monte Carlo prediction: unbiased but may have high variance and requires episodes or truncation.

Adaptive dynamic programming

Learn a model $\hat{P}, \hat{R}$ from experience, then solve the estimated MDP by dynamic programming. It is model-based RL:

1. count transitions and rewards,
2. estimate $\hat{P}, \hat{R}$,
3. run value iteration or policy iteration on the learned model.

Temporal difference learning

TD learns from bootstrapped one-step targets:

$$V(s) \leftarrow V(s) + \alpha (r + \gamma V(s') - V(s)).$$

TD combines Monte Carlo sampling with dynamic programming bootstrapping.

Exploration vs. exploitation

The agent must try actions to learn their consequences, but also use current knowledge to obtain reward.

Common strategies:

- ϵ -greedy,
- softmax/Boltzmann action selection,
- optimistic initialization,
- UCB-style exploration bonus,
- posterior sampling.

Q-learning

Off-policy TD control:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right).$$

It learns the greedy optimal value function independent of the behavior policy, assuming sufficient exploration and suitable learning rates.

Example

Let $Q(s, a) = 5$, reward $r = 2$, $\gamma = 0.9$, best next value $\max_{a'} Q(s', a') = 10$, and $\alpha = 0.1$. Target is $2 + 0.9 \cdot 10 = 11$. Update is $5 + 0.1(11 - 5) = 5.6$.

SARSA

On-policy TD control:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a)),$$

where a' is the next action actually selected by the current behavior policy.

Difference from Q-learning:

- Q-learning target uses best possible next action.
- SARSA target uses actually chosen next action, so it accounts for exploratory behavior.

Exam tip

Q-learning is off-policy: it learns the greedy optimal policy even while behaving exploratorily. SARSA is on-policy: it learns the value of the policy it is actually following, including its exploratory moves.

Policy search

Instead of learning values, directly optimize parameterized policy π_θ :

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \gamma^t R_{t+1} \right].$$

Methods:

- finite-difference/random search,
- policy gradient,
- REINFORCE,
- actor-critic.

Policy search is useful for continuous actions and when value maximization is hard, but can have high variance and local optima.

Exam Checklist

- Define PAC learning and online learning and compare assumptions.
- State VC dimension and why it matters.
- Learn conjunctions/disjunctions by elimination and by reductions.
- Define bandit regret and explain epsilon-greedy, UCB, Thompson sampling.
- Define MDP, value functions, optimal policy, Bellman equation.
- Explain value iteration, Monte Carlo/direct utility estimation, ADP, TD.
- Compare Q-learning and SARSA.
- Explain exploration/exploitation and policy search.

AI Specialization 2: LUP

Resolution in first-order logic, automatic proving, Boolean and predicate proving, and model search in generic domains.

Sources used: official CTU CourseWare page for BE4M36LUP, which lists lectures on propositional logic, SAT solving, Prolog, Herbrand models, SLD resolution, FOL, model finding, equality, paramodulation, subsumption, and superposition; CTU state-exam topic list; standard logic programming and automated-reasoning material.

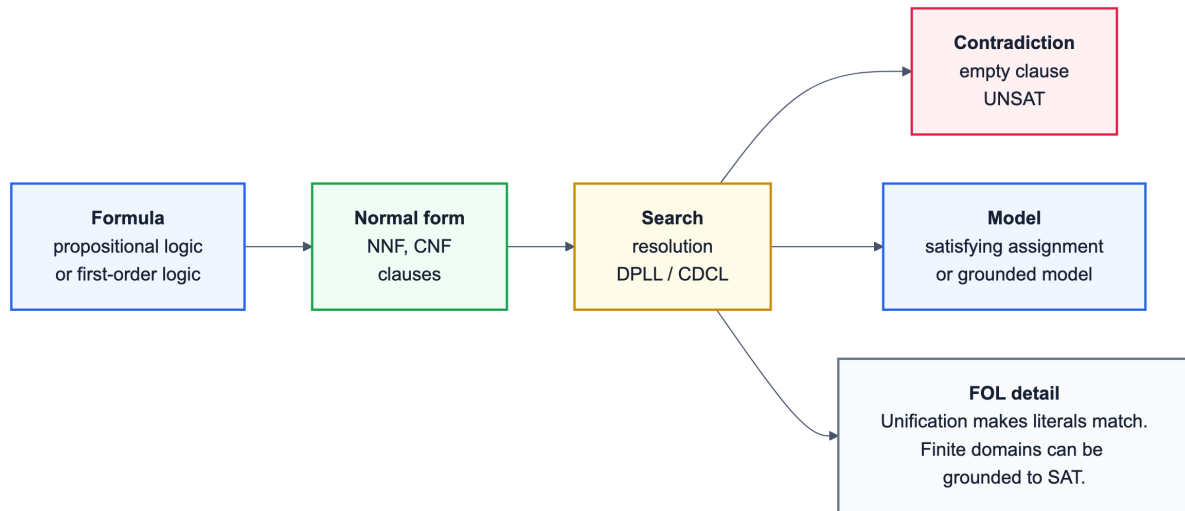


Figure 3: Automatic proving pipeline: formulas are normalized into clauses, then searched for a model or contradiction.

LUP in Plain English

LUP is about proving by search. Convert formulas into a form a machine can search, then either find a model or derive a contradiction.

1. Normalize.

CNF, clauses, Skolem form.

2. Search.

DPLL/CDCL, SLD, resolution, saturation.

3. Interpret result.

Model found, unsatisfiable, or proof by contradiction.

SAT and CNF

Boolean proving becomes clause search.

- CNF = conjunction of clauses.
- Tseitin avoids exponential CNF blowup.
- Resolution derives new clauses until contradiction.

DPLL/CDCL

Modern SAT solving is smart backtracking.

- Unit propagation forces assignments.
- Backtracking explores choices.
- Clause learning records why conflict happened.

Prolog and SLD

Logic programs answer goals by resolution.

- Horn clauses have at most one positive literal.
- SLD resolves selected goal with program rules.
- Negation as failure means "not provable".

First-order resolution

Variables require unification and Skolemization.

- Skolemization removes existential variables.
- Unification finds substitutions that make terms equal.
- Saturation keeps deriving clauses until done or contradiction.

Equality and models

Equality needs special handling; model search grounds domains.

- Paramodulation/superposition reason with equality.
- Grounding finite domains reduces to SAT.
- Subsumption removes redundant clauses.

How to answer

Say what representation the solver searches.

- Formula to clauses.
- Rules for deriving consequences.
- Stopping condition: model or empty clause.

Exam sentence: Resolution proves unsatisfiability by deriving the empty clause; in first-order logic this needs unification so clauses can match after substitution.

Three traps

- CNF conversion and Tseitin encoding are not the same thing.
- Skolemization preserves satisfiability, not logical equivalence.
- Negation as failure is procedural, not classical negation.

Use this page to keep the proving pipelines straight.

1. Propositional Normal Forms and SAT

Normal forms

Propositional formulas are built from variables, connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$.

Negation normal form (NNF): negations occur only in front of variables. Convert by eliminating $\Rightarrow, \Leftrightarrow$ and pushing negations inward with De Morgan rules.

Conjunctive normal form (CNF):

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each clause C_i is a disjunction of literals.

Disjunctive normal form (DNF): disjunction of conjunctions of literals.

For SAT solving, CNF is central. Naive distributive CNF conversion may blow up exponentially; Tseitin transformation introduces auxiliary variables and produces an equisatisfiable CNF of linear size.

SAT problem

SAT asks whether a propositional formula has a satisfying truth assignment. CNF-SAT is NP-complete. A clause is satisfied if at least one of its literals is true.

Resolution rule

For clauses:

$$(A \vee x), \quad (B \vee \neg x)$$

derive resolvent:

$$A \vee B.$$

Resolution is sound: every assignment satisfying the premises satisfies the resolvent. It is refutation-complete for propositional logic: a CNF is unsatisfiable iff repeated resolution can derive the empty clause $\square$.

To prove entailment $\Gamma \models \phi$, show $\Gamma \wedge \neg\phi$ is unsatisfiable.

Simple explanation

Resolution is a proof by contradiction. Add the negation of what you want to prove, then simplify clauses until contradiction appears. The empty clause means “there is no way to make all of this true at once.”

Example

From clauses $(P \vee Q)$ and $(\neg P \vee R)$, resolve on P to derive $(Q \vee R)$. If later you also have $(\neg Q)$ and $(\neg R)$, repeated resolution can derive the empty clause, proving inconsistency.

Unit propagation

A unit clause contains one literal, e.g. (x) . If a CNF contains (x) :

- clauses containing x are satisfied and removed,
- literal $\neg x$ is removed from all remaining clauses.

If an empty clause appears, conflict is detected. Unit propagation is the core inference step in DPLL/CDCL SAT solvers.

DPLL

DPLL is recursive backtracking with propagation:

1. Apply unit propagation.
2. Apply pure literal elimination if desired.
3. If all clauses satisfied, return SAT.
4. If empty clause, return UNSAT.
5. Pick unassigned variable and branch on true/false.

Clause learning and CDCL

Conflict-Driven Clause Learning (CDCL) improves DPLL:

- maintain implication graph from decisions and unit propagations,
- when conflict occurs, analyze it,
- derive a learned clause preventing the same conflict,
- backjump non-chronologically,
- use heuristics such as VSIDS and restarts.

Learned clauses are logical consequences of the formula, usually derived by resolution.

SMT and lazy approach

Satisfiability Modulo Theories (SMT) extends SAT with background theories, e.g. linear arithmetic, arrays, bit-vectors, equality with uninterpreted functions.

Lazy SMT architecture:

1. Replace theory atoms by Boolean variables.
2. SAT solver proposes a Boolean assignment.
3. Theory solver checks whether selected theory literals are consistent.
4. If inconsistent, theory solver returns a conflict clause.
5. SAT solver learns it and continues.

This combines efficient Boolean search with specialized theory reasoning.

Simple explanation

SAT handles the yes/no structure, while the theory solver handles meanings such as arithmetic or arrays. The SAT solver may suggest a Boolean combination that looks possible, and the theory solver says whether it is actually possible in the intended theory.

2. Logic Programming and Prolog

Definite and Horn clauses

A Horn clause has at most one positive literal. A definite clause has exactly one positive literal:

$$H \leftarrow B_1, \dots, B_n.$$

In logic form:

$$B_1 \wedge \dots \wedge B_n \Rightarrow H.$$

Facts have empty body: H . Queries ask whether a goal follows from the program.

Herbrand universe, base, and interpretations

For a first-order language:

- **Herbrand universe:** all ground terms constructible from constants and function symbols.
- **Herbrand base:** all ground atoms over predicate symbols and Herbrand universe.
- **Herbrand interpretation:** subset of the Herbrand base; atoms in it are true.

For definite programs, there is a unique least Herbrand model, obtainable as the least fixed point of the immediate consequence operator T_P .

Minimal model semantics

For definite logic programs, the declarative meaning is the least model. Operationally, SLD resolution computes answers soundly and completely with respect to this model, under fair search conditions.

SLD resolution

SLD = Selective Linear Definite-clause resolution.

Given goal:

$$\leftarrow G_1, \dots, G_k,$$

select atom G_i , unify it with the head H of some program clause

$$H \leftarrow B_1, \dots, B_m,$$

and replace G_i by $B_1, \dots, B_m$ under the most general unifier (MGU).

Prolog uses depth-first, left-to-right SLD search.

Exam tip

Prolog's operational behavior matters. The logical program may be declaratively correct, but depth-first left-to-right search can still loop or miss answers if predicates are ordered badly.

Example

Program: `parent(alice,bob). parent(bob,cara). ancestor(X,Y) :- parent(X,Y). ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).` Query `ancestor(alice,cara)` first tries direct parent and fails, then finds `Z=bob`, then proves `ancestor(bob,cara)` by the direct parent rule.

Negation as failure

Prolog's `not G` or `\\+ G` succeeds if Prolog fails to prove G .

This is not classical negation. It is based on the closed-world assumption: what cannot be proved is treated as false.

It is sound in well-behaved cases, especially for ground goals under stratified programs. With variables, it can be unintuitive:

`\\+ p(X)`

means “Prolog cannot prove any $p(X)$ ”, not “find X such that $p(X)$ is false”.

SLDNF resolution

SLDNF extends SLD with negation as failure. For negative literal $\neg A$:

- if A finitely fails, $\neg A$ succeeds,
- if A succeeds, $\neg A$ fails,
- if A loops, behavior may be non-terminating.

Cut

Prolog cut `!` commits to choices made since entering the current predicate.

Uses:

- improve efficiency by pruning alternatives,
- encode if-then-else,
- enforce determinism.

Green cut does not change declarative meaning; red cut changes meaning and must be used carefully.

3. First-Order Logic Normal Forms and Resolution

FOL normal-form pipeline

To prepare a FOL formula for resolution:

1. Eliminate $\Rightarrow, \Leftrightarrow$.
2. Move negations inward to NNF.
3. Standardize variables apart.
4. Move quantifiers to front: prenex normal form.
5. Skolemize existential quantifiers.
6. Drop universal quantifiers.
7. Convert matrix to CNF.
8. Split conjunction into clauses.

Skolemization preserves satisfiability, not equivalence. Example:

$$\forall x \exists y P(x, y)$$

becomes

$$\forall x P(x, f(x)).$$

Simple explanation

Skolemization replaces “there exists something” by a symbolic witness. If the witness depends on a universal variable, use a Skolem function like $f(x)$; if it does not depend on anything, use a Skolem constant.

Example

$\forall x \exists y \text{ Loves}(x, y)$ becomes $\forall x \text{ Loves}(x, f(x))$. The function f means “the thing loved by x .” By contrast, $\exists y \forall x \text{ Loves}(x, y)$ becomes $\forall x \text{ Loves}(x, c)$ because the witness does not depend on x .

Unification

FOL resolution needs unification because literals may contain variables and terms.

A substitution θ maps variables to terms. It unifies terms s, t if $s\theta = t\theta$.

The most general unifier (MGU) is a unifier from which all other unifiers can be obtained by further substitution.

Occurs check prevents invalid substitutions such as $x = f(x)$.

FOL resolution rule

Given clauses:

$$(A \vee L), \quad (B \vee \neg K)$$

if L and K have MGU θ , derive

$$(A \vee B)\theta.$$

Resolution is refutation-complete for first-order logic: if a set of clauses is unsatisfiable, saturation by resolution can derive $\square$ under a fair strategy.

Subsumption

Clause C subsumes clause D if there is substitution θ such that

$$C\theta \subseteq D.$$

Then D is redundant: C is at least as general. Removing subsumed clauses helps control saturation.

Example:

$$P(x) \text{ subsumes } P(a) \vee Q(b).$$

Saturation procedure

Saturation repeatedly adds logical consequences until:

- the empty clause is derived: unsatisfiable,
- no new non-redundant clauses can be generated: saturated set.

Generic loop:

1. maintain active and passive clause sets,
2. select a passive clause,
3. generate inferences with active clauses,
4. simplify by subsumption/rewriting,
5. add non-redundant results.

Saturation may not terminate for satisfiable first-order problems.

4. Equality in First-Order Logic

Axiomatic equality

One approach adds equality axioms:

- reflexivity: $x = x$,
- symmetry: $x = y \Rightarrow y = x$,
- transitivity: $x = y \wedge y = z \Rightarrow x = z$,
- congruence for functions:

$$x_i = y_i \ \forall i \Rightarrow f(x_1, \dots, x_n) = f(y_1, \dots, y_n),$$

- substitutivity for predicates.

Problem: generates many clauses and is inefficient.

Paramodulation and superposition

Instead of axiomatizing equality fully, extend resolution with equality inference.

Paramodulation idea:

from equality $s = t$ and a clause containing subterm s' unifiable with s , replace that subterm by t under the unifier.

Superposition is a refined, ordered version used in modern theorem provers. It combines:

- resolution,
- equality reasoning,
- term ordering,
- simplification by rewriting.

This is much more efficient than explicit equality axioms.

5. Searching for Models in Generic Domains

Grounding

A first-order problem over a finite domain can be grounded:

1. choose finite domain objects,
2. instantiate variables by all domain objects,
3. replace each ground atom by a propositional variable,
4. encode constraints as SAT.

Example:

$$\forall x P(x) \vee Q(x)$$

over domain $\{a, b\}$ becomes

$$(P_a \vee Q_a) \wedge (P_b \vee Q_b).$$

Propositional encoding

Common constraints:

- at least one: $x_1 \vee \dots \vee x_n$,
- at most one: $\neg x_i \vee \neg x_j$ for all pairs,
- exactly one: at least one plus at most one,
- implication: $a \Rightarrow b$ becomes $\neg a \vee b$.

Model search becomes SAT solving. This is used in planning, scheduling, puzzles, bounded verification, and finite model finding.

Grounding challenges

Grounding can explode combinatorially. If a predicate has arity k and domain size n , it has n^k ground atoms.

Mitigation:

- typed domains,
- symmetry breaking,
- constraint propagation before grounding,
- lazy grounding,
- compact cardinality encodings,
- incremental SAT.

Exam Checklist

- Convert formulas to NNF/CNF and explain Tseitin transformation.
- State resolution, unit propagation, DPLL, CDCL, and clause learning.
- Explain lazy SMT.
- Define Horn/definite clauses, Herbrand interpretations, and minimal model semantics.
- Explain SLD, SLDNF, negation as failure, and cut.
- Convert FOL to clause form and explain Skolemization.
- Explain unification, MGU, subsumption, and saturation.
- Compare equality axioms with paramodulation/superposition.
- Explain grounding and SAT encoding for model search.

AI Specialization 3: SSU

Minimizing empirical risk. Maximum likelihood estimation, EM algorithm. Classical and deep neural networks and their learning.

Sources used: official CTU CourseWare page for BE4M33SSU lectures, which lists empirical risk, empirical risk minimization, SVMs, supervised learning for deep networks, SGD/deep convolutional networks, and generative learning/EM; CTU state-exam topic list; standard statistical learning material.

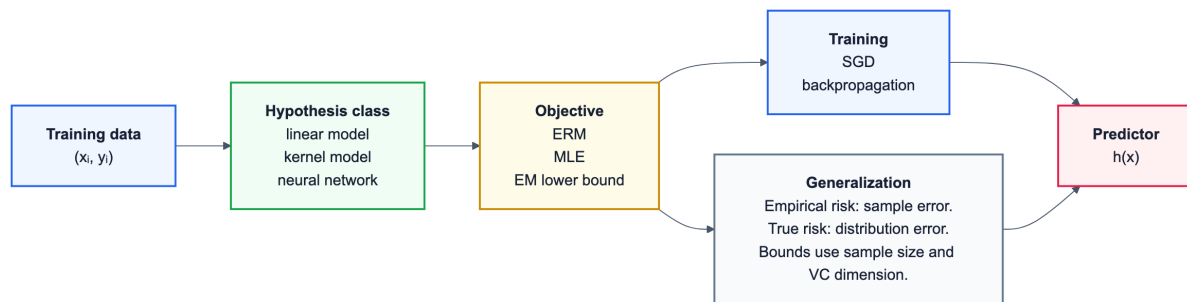


Figure 4: Supervised-learning pipeline: data and a hypothesis class define an objective, training optimizes it, and generalization controls performance on unseen data.

SSU in Plain English

SSU is about learning a predictor from data. The big story is: define loss, minimize training loss, control generalization, and understand the main model families.

1. Define the objective.

Risk, empirical risk, likelihood, or loss.

2. Pick the model.

SVM, probabilistic model, EM, neural network.

3. Explain training.

Optimization plus generalization.

ERM and generalization

Training error is not the same as true error.

- Risk = expected loss on the real distribution.
- Empirical risk = average loss on the sample.
- Bounds explain when low training loss transfers.

VC dimension

A capacity measure for hypothesis classes.

- High capacity can fit more patterns.
- Too much capacity risks overfitting.
- Consistency means convergence to optimal risk.

SVMs

Find a separating hyperplane with maximum margin.

- Hard margin: separable data.
- Soft margin: allows violations with penalty.
- Kernels make nonlinear boundaries via inner products.

MLE and EM

Fit probabilistic models by maximizing likelihood.

- MLE chooses parameters that make data likely.
- EM handles hidden variables.
- E-step estimates hidden assignments; M-step updates parameters.

Deep networks

Layered differentiable functions trained by gradients.

- Backprop computes gradients through layers.
- SGD updates parameters from mini-batches.
- CNNs use convolution for spatial structure.

How to answer

Tie every method to loss and optimization.

- What is minimized or maximized?
- What are the parameters?
- How do we avoid overfitting?

Exam sentence: ERM minimizes training loss, but learning theory asks when this also gives low true risk; VC dimension and concentration bounds control that gap.

Three traps

- Empirical risk is measured on the sample; true risk is an expectation over the unknown distribution.
- EM increases a likelihood lower bound, not by magic "guessing".
- Backprop computes gradients; SGD is the optimizer using them.

Use this page to connect formulas to the learning story.

1. Empirical Risk Minimization

Prediction setup

Let examples (x, y) be drawn from unknown distribution $P(X, Y)$. A predictor $h : X \rightarrow Y$ is chosen from hypothesis class H . A loss function $\ell(h(x), y)$ measures error.

Examples:

- 0/1 classification loss: $\ell(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$.
- Squared regression loss: $\ell(\hat{y}, y) = (\hat{y} - y)^2$.
- Hinge loss: $\ell(yf(x)) = \max(0, 1 - yf(x))$.
- Logistic loss: $\ell(yf(x)) = \log(1 + \exp(-yf(x)))$.

Risk and empirical risk

True risk:

$$R(h) = \mathbb{E}_{(x,y) \sim P}[\ell(h(x), y)].$$

Empirical risk on sample $S = \{(x_i, y_i)\}_{i=1}^m$:

$$\hat{R}_S(h) = \frac{1}{m} \sum_{i=1}^m \ell(h(x_i), y_i).$$

Empirical risk minimization (ERM):

$$\hat{h} \in \arg \min_{h \in H} \hat{R}_S(h).$$

ERM works when empirical risk uniformly approximates true risk over H .

Hoeffding inequality

If $Z_1, \dots, Z_m$ are independent random variables in $[0, 1]$ with mean μ , then

$$\Pr \left[\left| \frac{1}{m} \sum_i Z_i - \mu \right| \geq \epsilon \right] \leq 2e^{-2m\epsilon^2}.$$

For a fixed hypothesis h , this bounds deviation between $\hat{R}_S(h)$ and $R(h)$.

For finite H , union bound gives:

$$\Pr [\exists h \in H : |\hat{R}_S(h) - R(h)| > \epsilon] \leq 2|H|e^{-2m\epsilon^2}.$$

Thus with probability at least $1 - \delta$:

$$\forall h \in H : R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\ln(2|H|/\delta)}{2m}}.$$

Generalization bounds

A generalization bound upper-bounds true risk using empirical risk plus a complexity term:

$$R(h) \leq \hat{R}_S(h) + \text{complexity}(H, m, \delta).$$

For infinite classes, VC dimension replaces $\ln |H|$. If $VC(H) = d$, typical rates scale like

$$O\left(\sqrt{\frac{d + \ln(1/\delta)}{m}}\right).$$

The message: low training error is not enough; model class complexity matters.

Simple explanation

Empirical risk is training error. True risk is real-world error. Generalization bounds explain when training error is trustworthy: more data and simpler hypothesis classes make the gap between training and real-world performance smaller.

Example

If a classifier makes 3 mistakes on 100 training examples, its empirical 0/1 risk is 0.03. This does not prove the real-world error is exactly 3%; a generalization bound adds a safety margin depending on sample size and hypothesis complexity.

Statistical consistency

A learning algorithm is statistically consistent if, as $m \rightarrow \infty$, its risk converges to the best achievable risk in the considered class or to Bayes risk.

ERM is consistent under conditions such as:

- hypothesis class has finite VC dimension,
- uniform convergence holds,
- optimization solves or approximates ERM,
- the class is rich enough if consistency is meant relative to Bayes risk.

VC dimension

VC dimension is the largest number of points that can be shattered by H . It measures expressive power for binary classification.

Examples:

- Intervals on the real line: VC dimension 2.
- Linear classifiers in $\mathbb{R}^d$: VC dimension $d + 1$.

High VC dimension permits fitting many labelings and increases overfitting risk.

2. Support Vector Machines

Linear separable SVM

Binary labels $y_i \in \{-1, +1\}$, classifier

$$f(x) = w^T x + b.$$

Correct classification with margin:

$$y_i(w^T x_i + b) \geq 1.$$

Margin is $2/\|w\|$. Maximizing margin equals minimizing $\frac{1}{2}\|w\|^2$:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1.$$

Soft-margin SVM

Allow violations with slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Equivalent regularized ERM with hinge loss:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b)).$$

Parameter C trades margin size against training errors.

Kernel SVM

The dual form depends on dot products $x_i^T x_j$. Replace them by kernel

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

without explicitly constructing ϕ .

Common kernels:

- polynomial: $K(x, z) = (x^T z + c)^p$,
- RBF/Gaussian: $K(x, z) = \exp(-\|x - z\|^2 / (2\sigma^2))$.

Prediction:

$$f(x) = \sum_i \alpha_i y_i K(x_i, x) + b.$$

Only support vectors have $\alpha_i > 0$.

Simple explanation

An SVM is mostly determined by the borderline examples. Points far away from the separating boundary do not affect the final classifier; the support vectors are the points that hold the margin in place.

Example

In 2D, imagine two classes separated by a line. If you move a point far inside its class region, the best separating line usually does not move. If you move a point lying on the margin, the line may change; that point is a support vector.

3. Maximum Likelihood and EM

Maximum likelihood estimator

Given data $D = \{x_i\}_{i=1}^m$ and parametric model $p_\theta(x)$, likelihood:

$$L(\theta) = \prod_{i=1}^m p_\theta(x_i).$$

Log-likelihood:

$$\ell(\theta) = \sum_{i=1}^m \log p_\theta(x_i).$$

MLE:

$$\hat{\theta}_{MLE} \in \arg \max_{\theta} \ell(\theta).$$

Examples:

- Bernoulli MLE: $\hat{p} = \frac{1}{m} \sum_i x_i$.
- Gaussian mean MLE: sample mean.

Consistency of an estimator

Estimator $\hat{\theta}_m$ is consistent for θ^* if

$$\hat{\theta}_m \xrightarrow{P} \theta^*$$

as $m \rightarrow \infty$. MLE is consistent under regularity and identifiability conditions: correct model family, enough data, identifiable parameters, and suitable smoothness.

Latent-variable likelihood

For latent variables z :

$$p_\theta(x) = \sum_z p_\theta(x, z)$$

or integral for continuous z . Direct maximization of

$$\sum_i \log \sum_z p_\theta(x_i, z)$$

is often hard because of the log of a sum.

EM lower bound

For any distribution $q_i(z)$:

$$\log p_\theta(x_i) = \log \sum_z q_i(z) \frac{p_\theta(x_i, z)}{q_i(z)} \geq \sum_z q_i(z) \log \frac{p_\theta(x_i, z)}{q_i(z)}$$

by Jensen's inequality.

Define lower bound:

$$\mathcal{L}(q, \theta) = \sum_i \sum_z q_i(z) \log p_\theta(x_i, z) - \sum_i \sum_z q_i(z) \log q_i(z).$$

The gap to log-likelihood is a KL divergence:

$$\log p_\theta(D) - \mathcal{L}(q, \theta) = \sum_i KL(q_i(z) \| p_\theta(z|x_i)).$$

EM algorithm

EM performs block-coordinate ascent on the lower bound. Equivalently, if the objective is written as negative log-likelihood or negative lower bound, the same procedure can be described as block-coordinate descent.

E-step:

$$q_i(z) \leftarrow p_{\theta^{old}}(z|x_i).$$

This makes the lower bound tight at current parameters.

M-step:

$$\theta^{new} \leftarrow \arg \max_{\theta} \sum_i \mathbb{E}_{z \sim q_i} [\log p_\theta(x_i, z)].$$

EM monotonically increases likelihood, but may converge to a local optimum.

Simple explanation

EM alternates between guessing hidden variables and refitting parameters. In a Gaussian mixture, the E-step softly assigns points to clusters; the M-step recomputes cluster means, variances, and weights from those soft assignments.

Example

For a two-Gaussian mixture and one data point x , the E-step might compute responsibilities 0.8 for cluster 1 and 0.2 for cluster 2. In the M-step, x contributes 80% of its weight to updating cluster 1's mean and 20% to cluster 2's mean.

Typical applications:

- Gaussian mixture models,

- hidden Markov models,
- missing data,
- mixture of experts.

4. Deep Networks and Their Training

Neuron

A neuron computes:

$$z = w^T x + b, \quad a = \sigma(z),$$

where σ is activation function.

Common activations:

- sigmoid: $\sigma(z) = 1/(1 + e^{-z})$,
- tanh,
- ReLU: $\max(0, z)$,
- leaky ReLU,
- softmax for multiclass output.

Network architectures

Feedforward neural network:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}).$$

Architectural types:

- MLP: fully connected layers.
- CNN: convolution, pooling, local receptive fields.
- RNN/LSTM/GRU: sequence models.
- Residual networks: skip connections to ease optimization.
- Autoencoders: learn latent representation by reconstruction.

Convolutional networks

Convolutional layer applies learned filters over spatial positions:

$$y_{k,i,j} = \sum_{c,u,v} W_{k,c,u,v} x_{c,i+u,j+v} + b_k.$$

Key ideas:

- local connectivity,
- weight sharing,
- translation equivariance,
- pooling/striding for spatial downsampling,
- many channels/features.

CNNs are standard for images and spatial signals.

Backpropagation

Backprop computes gradients by chain rule from output layer backward.

Simple explanation

Backpropagation is bookkeeping for the chain rule. First compute how wrong the output is, then move backward layer by layer, asking how much each weight contributed to that error.

For layer:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)}).$$

Error signal:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}.$$

Gradients:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}(a^{(l-1)})^T, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

Backward recursion:

$$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)}).$$

Layer types

Common layers:

- dense/fully connected,
- convolution,
- pooling,
- normalization, e.g. batch normalization,
- dropout,
- recurrent layers,
- residual/skip connections,
- embedding layers.

Parameter initialization

Bad initialization can cause vanishing/exploding activations and gradients.

Common schemes:

- Xavier/Glorot for tanh/sigmoid-like activations:

$$\text{Var}(W) \approx \frac{2}{n_{in} + n_{out}}.$$

- He initialization for ReLU:

$$\text{Var}(W) \approx \frac{2}{n_{in}}.$$

Biases are often initialized to zero or small constants.

Stochastic gradient descent

Full gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

SGD estimates gradient from minibatch B :

$$\theta \leftarrow \theta - \eta \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta).$$

Practical improvements:

- momentum,
- Adam/RMSProp,
- learning-rate schedules,
- weight decay,
- early stopping,
- data augmentation,
- normalization.

Overfitting and generalization in deep networks

Deep networks can overfit because they have many parameters. Regularization methods:

- explicit penalties: L_2 /weight decay,
- dropout,
- data augmentation,
- early stopping,
- batch normalization effects,
- architecture constraints.

Even overparameterized networks can generalize well, but exam answers should emphasize training/validation split and regularization.

Exam Checklist

- Define risk, empirical risk, ERM.
- State Hoeffding inequality and finite-class generalization bound.
- Explain consistency and VC dimension.
- Formulate hard/soft-margin SVM and kernel trick.
- Define MLE and consistency.
- Derive EM as lower-bound maximization with E-step and M-step.
- Explain neurons, MLPs, CNNs, backpropagation, layer types.
- Explain initialization and SGD/minibatch training.

AI Specialization 4: PUI

Domain independent planning: representations, heuristics, algorithms, nondeterministic and probabilistic planning, MDPs, and MCTS.

Sources used: official CTU CourseWare page for BE4M36PUI, which lists lectures on classical planning, state-space search, relaxation heuristics, landmarks/LM-cut, LP heuristics, abstrac-

tion heuristics, nondeterministic planning, probabilistic planning, MDPs/POMDPs, and MCTS; CTU state-exam topic list; standard automated planning material.

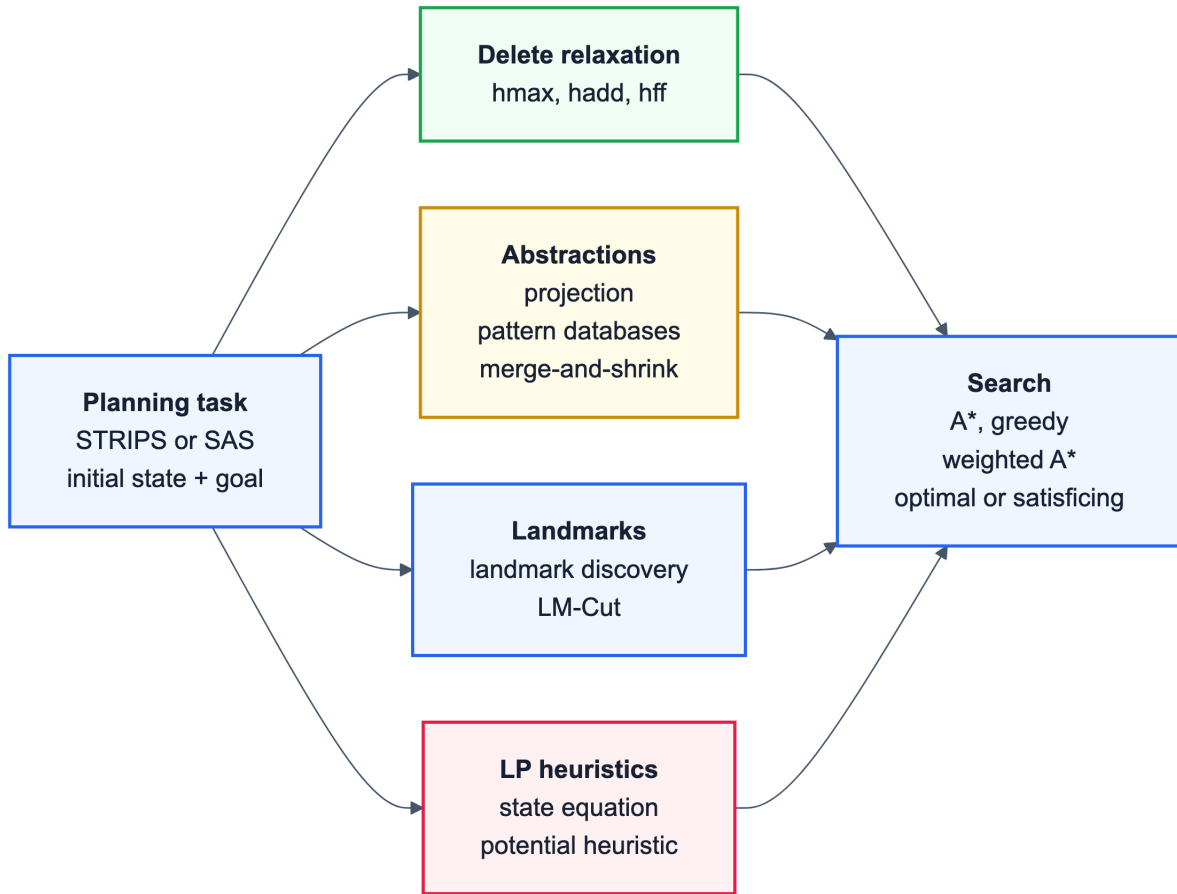


Figure 5: Planning heuristic families: different relaxations and abstractions estimate distance-to-goal and guide heuristic search.

PUI in Plain English

PUI is about planning actions before acting. The main trick is to estimate distance to the goal using relaxations, abstractions, landmarks, or optimization.

1. State the planning model.

States, actions, initial state, goal.

2. Choose the heuristic.

Relaxed, abstracted, landmark, LP, or sampled.

3. Say the guarantee.

Satisficing, optimal, admissible, probabilistic.

Classical planning

Find an action sequence from initial state to goal.

- STRIPS: facts, preconditions, add/delete effects.
- SAS: finite-domain variables.
- A*, greedy search, and heuristic search are central.

Delete relaxation

Pretend actions never delete facts.

- hmax: optimistic max cost.
- hadd: sums subgoal costs, may double count.
- hff: extracts a relaxed plan.

Abstractions and landmarks

Simplify the problem or identify required facts/actions.

- Pattern databases solve projected tasks.
- Merge-and-shrink builds abstract transition systems.
- Landmarks must be achieved on every plan.

LP heuristics

Use linear constraints to lower-bound plan cost.

- State-equation heuristic counts action occurrences.
- Potential heuristics assign values to facts/states.
- LP gives a numeric estimate for search.

Uncertainty and MCTS

Actions may be nondeterministic or probabilistic.

- MDP value iteration optimizes expected return.
- MCTS samples futures instead of expanding all states.
- UCT balances value and exploration.

How to answer

Always say what information is ignored or approximated.

- What is the relaxed problem?
- Is the heuristic admissible?
- What search uses it?

Exam sentence: Delete-relaxation heuristics estimate goal distance by solving an easier problem where delete effects are ignored, so facts only accumulate.

Three traps

- Satisficing means "find some plan"; optimal means minimum cost.
- hadd is usually not admissible because it double-counts shared actions.
- MCTS is useful when the full search tree is too large to enumerate.

Use this page to remember what each heuristic family is simplifying.

1. Planning Tasks and Heuristic Search

Classical planning

A classical planning task assumes:

- finite set of states,
- deterministic actions,
- fully observable state,
- known initial state,
- goal condition,
- static domain.

A plan is a sequence of actions transforming the initial state into a goal state.

STRIPS representation

STRIPS uses propositional facts. Task:

$$\Pi = (F, A, I, G)$$

where:

- F facts,
- A actions,
- $I \subseteq F$ initial facts,
- $G \subseteq F$ goal facts.

Action a has:

- preconditions $pre(a)$,
- add effects $add(a)$,
- delete effects $del(a)$.

Action applicable in state s iff $pre(a) \subseteq s$. Result:

$$\gamma(s, a) = (s \setminus del(a)) \cup add(a).$$

SAS representation

SAS uses finite-domain variables:

$$V = \{v_1, \dots, v_n\}, \quad D(v_i) \text{ finite.}$$

A state assigns one value to every variable. Actions have partial variable assignments as preconditions/effects.

SAS can be more compact than STRIPS because mutually exclusive facts become values of one variable.

Satisficing vs. optimal planning

- **Satisficing planning:** find any valid plan, usually fast.
- **Optimal planning:** find plan with minimum cost or length; requires admissible search to prove optimality.

Heuristic search

Planning is often modeled as graph search in state space.

Common algorithms:

- BFS for unit costs,
- uniform-cost search for positive costs,
- greedy best-first search with priority $h(s)$,
- A* with priority $f(s) = g(s) + h(s)$.

Heuristic properties

A heuristic $h(s)$ estimates cost from state s to goal.

- **Admissible:** $h(s) \leq h^*(s)$ for all s .
- **Consistent:** $h(s) \leq c(s, a, s') + h(s')$ for every transition.
- **Safe/dead-end aware:** returns ∞ for recognized dead ends.
- **Informative:** closer to h^* is usually better.

A* with admissible heuristic is optimal; consistency avoids reopening closed nodes.

Simple explanation

A planning heuristic is like a GPS estimate of remaining distance. If it never overestimates, A* can trust it for optimal planning. If it is too optimistic but informative, it can still be excellent for quickly finding some plan.

2. Delete Relaxation and Abstraction Heuristics

Delete relaxation

Delete relaxation ignores delete effects:

$$del(a) = \emptyset.$$

The relaxed problem is easier because facts only accumulate. It may allow impossible plans, so its optimal cost is an admissible lower bound for the original task.

h^{max}

Estimate cost of achieving facts:

$$h^{max}(s, p) = \begin{cases} 0 & p \in s, \\ \min_{a: p \in add(a)} \left(cost(a) + \max_{q \in pre(a)} h^{max}(s, q) \right) & otherwise. \end{cases}$$

Goal value:

$$h^{max}(s) = \max_{g \in G} h^{max}(s, g).$$

Admissible but often weak because it assumes subgoals can be achieved together for the maximum cost.

h^{add}

Use sum instead of maximum:

$$h^{add}(s, p) = \min_{a: p \in add(a)} \left(cost(a) + \sum_{q \in pre(a)} h^{add}(s, q) \right).$$

Goal value:

$$h^{add}(s) = \sum_{g \in G} h^{add}(s, g).$$

More informative but not admissible because it double-counts shared actions.

h^{ff}

Fast-Forward heuristic:

1. build relaxed planning graph,
2. extract one relaxed plan greedily backward from goals,
3. heuristic is number/cost of actions in that relaxed plan.

h^{ff} is not admissible, but works well in satisficing planning.

Simple explanation

Delete relaxation imagines that actions never undo facts. That fantasy world is easier: once a door is open, it stays open; once a package is loaded, it can still magically be somewhere else later. The resulting plan may be impossible, but its cost is useful guidance.

Example

In a logistics task, moving a truck from city A to city B normally deletes `at(truck,A)` and adds `at(truck,B)`. Delete relaxation keeps both facts. This can make relaxed plans overly optimistic, but it quickly estimates which actions are relevant.

Abstraction heuristics

An abstraction maps concrete states to abstract states:

$$\alpha : S \rightarrow S^\alpha.$$

Solve the abstract problem exactly and use abstract distance:

$$h^\alpha(s) = \text{dist}^\alpha(\alpha(s), G^\alpha).$$

If abstraction does not increase costs, h^α is admissible.

Projection and pattern databases

Projection abstraction keeps only selected variables (pattern P). The pattern database stores exact goal distances for all abstract states over P .

Advantages:

- admissible,
- very fast lookup after preprocessing.

Tradeoff: larger patterns are more informative but exponentially larger.

Merge & Shrink

Merge-and-shrink abstraction:

1. create atomic abstractions for variables,
2. iteratively merge abstractions,
3. shrink abstract state spaces to keep size manageable,
4. compute distances in final abstraction.

It can produce strong admissible heuristics while controlling memory.

3. Landmarks and LM-Cut

Landmarks

A landmark is a fact, action, or formula that must be true/achieved in every valid plan.

Example: if goal is `at(package, destination)`, then loading the package may be an action landmark in logistics-like domains.

Landmark orders:

- natural order: L_1 must be achieved before L_2 ,
- greedy necessary order: L_1 must hold immediately before first achieving L_2 ,
- reasonable order: not logically necessary but useful.

Landmark discovery

Common methods:

- relaxed planning graph analysis,
- backchaining from goals through achievers,
- checking necessary predecessors,
- disjunctive landmarks when one of several facts/actions must occur.

Landmarks can guide search by counting unachieved landmarks or by preferred operators.

Landmark heuristic

A simple landmark-count heuristic:

$$h(s) = \text{number or cost of landmarks not yet achieved from } s.$$

Must avoid double-counting and handle landmarks that can be deleted and re-achieved.

LM-Cut heuristic

LM-Cut is an admissible heuristic based on repeated cuts in a relaxed planning graph.

High-level algorithm:

1. Compute h^{max} values.
2. Identify a cut: a set of actions that every relaxed plan must use to cross from reachable facts to goal-relevant facts.
3. Let cut cost be minimum action cost in the cut.
4. Add this cost to heuristic.
5. Subtract cut cost from all actions in the cut.
6. Repeat until relaxed goal cost is zero.

LM-Cut is admissible and often much stronger than h^{max} .

Example

If every relaxed plan must use one of three actions to first achieve a required landmark, those actions form a cut. If their costs are 2, 5, and 7, LM-Cut can safely add 2 to the heuristic and subtract 2 from all actions in that cut.

Exam tip

Remember the roles: h^{max} gives a cheap admissible estimate, h^{add} is usually stronger but not admissible, h^{ff} extracts one relaxed plan, and LM-Cut repeatedly finds necessary relaxed-action cuts to stay admissible.

4. Linear Programming Heuristics

State-equation heuristic

Represent a relaxed plan by action-count variables $x_a \geq 0$. For each fact/variable-value, require that action effects can support net changes needed from state to goal.

Generic form:

$$Mx \geq b(s, G), \quad x \geq 0.$$

Minimize action cost:

$$h(s) = \min \sum_a cost(a)x_a.$$

The LP relaxation gives an admissible heuristic if constraints are necessary for real plans.

Potential heuristic

Assign potentials to facts or variable values. For state s :

$$h(s) = \sum_{p \in s} w_p$$

or for SAS:

$$h(s) = \sum_i w_{i, s_i}.$$

Weights are chosen so that for every action:

$$h(s) \leq cost(a) + h(\gamma(s, a)),$$

and goal states have heuristic ≤ 0 . This makes h admissible and consistent. Weights can be optimized by LP to maximize heuristic values over relevant states.

Planning portfolios

Many planners combine several heuristics/search strategies. A portfolio may run planners sequentially or in parallel with time budgets. This is practical because planner performance is highly domain-dependent.

5. Nondeterministic and Probabilistic Planning

Nondeterministic planning

Actions have several possible outcomes:

$$\gamma(s, a) \subseteq S.$$

A solution is generally a policy, not a sequence:

$$\pi : S \rightarrow A.$$

Plan types:

- **weak plan:** at least one execution reaches goal.
- **strong plan:** all executions reach goal in finite time.
- **strong cyclic plan:** all fair executions reach goal, allowing loops while progress is eventually possible.

Simple explanation

In nondeterministic planning, a sequence of actions is usually not enough because the world may choose different outcomes. A policy says what to do after each possible outcome, like a contingency plan.

Algorithms use AND/OR search:

- OR nodes: planner chooses action.
- AND nodes: environment chooses outcome.

Probabilistic planning and MDPs

Probabilistic planning assigns probabilities to outcomes:

$$P(s'|s, a).$$

This is an MDP:

$$(S, A, P, R, \gamma).$$

Goal: find policy maximizing expected discounted reward or minimizing expected cost.

Value iteration

For discounted MDP:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

For stochastic shortest-path planning, terminal goal states have value 0 and costs are minimized.

POMDPs

Partially observable MDPs maintain belief states $b(s)$, distributions over states. Actions update beliefs using Bayes rule after observations. POMDPs are much harder than fully observable MDPs.

Monte-Carlo Tree Search

MCTS builds an asymmetric search tree by simulation.

Each iteration:

1. **Selection:** descend tree using a tree policy.
2. **Expansion:** add a new node.
3. **Simulation:** roll out a default policy.
4. **Backpropagation:** update visit counts and values.

UCT selection rule:

$$a = \arg \max_a \left(\bar{X}(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}} \right).$$

First term exploits high-value actions; second explores uncertain actions. UCT is widely used in games and planning with generative models.

Example

Suppose action A has average rollout value 0.7 from 100 visits, while action B has 0.6 from only 4 visits. UCT may choose B because the exploration bonus for 4 visits is much larger, so B is still uncertain and worth testing.

Exam Checklist

- Define STRIPS and SAS and compare them.
- Explain satisficing vs. optimal planning.
- State admissible/consistent heuristic properties.
- Compute/describe h^{max} , h^{add} , and h^{ff} .
- Explain abstraction, projection, pattern databases, and merge-and-shrink.
- Define landmarks, landmark orders, and LM-Cut.
- Explain state-equation and potential heuristics.
- Distinguish weak, strong, and strong cyclic nondeterministic plans.
- Define MDP value iteration and MCTS/UCT.

AI Specialization 5: MAS

Autonomous agents and multiagent systems. Noncooperative game theory, extensive-form games, coalitional games, and auctions.

Sources used: official CTU CourseWare page for Computational Game Theory / BE4M36MAS, which describes the course as covering multiagent models and algorithms for cooperative and non-cooperative settings; CTU state-exam topic list; standard algorithmic game theory and multiagent systems material.

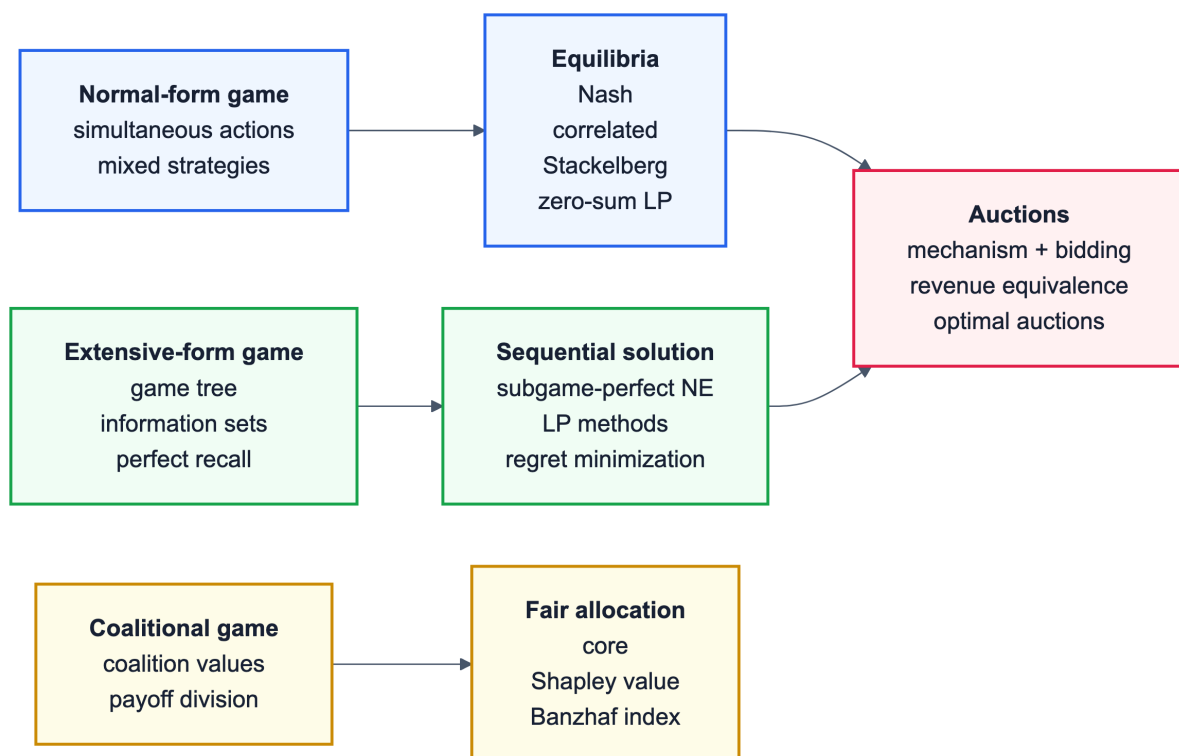


Figure 6: Game-theory map: each representation points to the solution concepts most commonly used for it.

MAS in Plain English

MAS is about strategic interaction. First identify the game representation, then the solution concept: equilibrium, value, coalition payoff, or auction outcome.

1. Identify players.

Who chooses, and what do they know?

2. Identify representation.

Normal form, extensive form, coalition, auction.

3. Name the solution.

Nash, minimax, core, Shapley, winner/prices.

Normal-form games

One-shot strategic choices.

- Players choose actions; utilities depend on action profile.
- Nash equilibrium: no player wants to deviate alone.
- Dominated strategies can be removed.

Zero-sum and Stackelberg

Special structures with clearer solution methods.

- Zero-sum: one player's gain is another's loss.
- Minimax can be solved by LP.
- Stackelberg: leader commits, follower responds.

Extensive-form games

Sequential decisions in a game tree.

- Information sets model imperfect information.
- Perfect recall means players remember own observations/actions.
- Subgame-perfect equilibrium refines Nash.

Coalitional games

Groups form and share value.

- Core: no coalition wants to break away.
- Shapley value: average marginal contribution.
- Voting indices measure power, not just votes.

Auctions

Mechanisms allocate goods and set payments.

- First-price and second-price incentives differ.
- Revenue equivalence needs assumptions.
- Combinatorial auctions handle bundles.

How to answer

Representation first, solution concept second.

- What choices are simultaneous or sequential?
- What can players observe?
- What stability or optimality notion is used?

Exam sentence: A Nash equilibrium is a strategy profile where each player's strategy is a best response to the others, so no single player can improve by deviating alone.

Three traps

- Nash equilibrium need not maximize total social welfare.
- Subgame-perfect equilibrium only applies cleanly to proper subgames.
- Shapley value is about marginal contribution, not equal splitting.

Use this page to pick the right game model before computing anything.

1. Normal-Form Games

Definition

A normal-form game is

$$G = (N, (A_i)_{i \in N}, (u_i)_{i \in N}),$$

where:

- N is set of players,
- A_i is action set of player i ,
- $u_i : A_1 \times \dots \times A_n \rightarrow \mathbb{R}$ is utility.

A pure strategy is an action $a_i \in A_i$. A mixed strategy is a probability distribution $\sigma_i \in \Delta(A_i)$.

Expected utility:

$$u_i(\sigma) = \sum_{a \in A} u_i(a) \prod_j \sigma_j(a_j).$$

Dominated strategies

Strategy s_i is strictly dominated by s'_i if

$$u_i(s'_i, a_{-i}) > u_i(s_i, a_{-i})$$

for all opponents' actions a_{-i} .

Weak dominance uses $\geq$ for all and $>$ for at least one case.

Iterated removal of strictly dominated strategies is order-independent. Weak dominance removal can depend on order.

Nash equilibrium

A mixed strategy profile σ^* is Nash equilibrium if no player can improve by unilateral deviation:

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \forall i, \sigma_i.$$

Properties:

- every finite game has at least one mixed Nash equilibrium,
- pure equilibrium may not exist,
- equilibrium need not be unique,
- equilibrium need not maximize social welfare.

Simple explanation

A Nash equilibrium is stable against one-player deviations. It does not mean the outcome is good for everyone; it only means no single player can improve by changing strategy while the others stay fixed.

Example

In Prisoner's Dilemma, both players defecting is a Nash equilibrium: if one player expects the other to defect, switching to cooperate makes them worse off. Yet both cooperating would be better for both, so Nash equilibrium is not the same as social optimum.

Computing Nash equilibrium

Two-player zero-sum games are polynomial-time solvable by LP.

General two-player games:

- support enumeration,
- Lemke-Howson algorithm,
- solving complementarity conditions.

Complexity: finding one Nash equilibrium in a finite normal-form game is PPAD-complete.

Two-person zero-sum games

Player 1 payoff matrix A , player 2 payoff $-A$.

Player 1 solves:

$$\max v$$

subject to:

$$\sum_i x_i A_{ij} \geq v \quad \forall j, \quad \sum_i x_i = 1, \quad x_i \geq 0.$$

Player 2 solves dual minimization. Minimax theorem:

$$\max_x \min_y x^T A y = \min_y \max_x x^T A y.$$

Correlated equilibrium

A mediator samples action profile a from distribution $p(a)$ and privately recommends a_i to player i . It is correlated equilibrium if no player wants to deviate after seeing recommendation:

$$\sum_{a_{-i}} p(a_i, a_{-i}) [u_i(a_i, a_{-i}) - u_i(a'_i, a_{-i})] \geq 0$$

for all i, a_i, a'_i . It is computable by linear programming.

Stackelberg equilibrium

Leader commits to strategy first; follower observes and best-responds. The leader chooses commitment maximizing utility under follower response.

Used in security games. Compared to Nash, commitment can benefit the leader because it changes follower incentives.

Computation is naturally bilevel: the leader optimizes while anticipating follower best responses. In finite games it can be formulated by enumerating follower best-response regions, by mathematical programming, or by specialized algorithms for security games.

Simple explanation

Stackelberg games are about credible commitment. If a defender publicly commits to a patrol distribution, an attacker best-responds to that distribution. The defender chooses the commitment knowing this reaction will happen.

2. Extensive-Form Games

Representation

An extensive-form game is a game tree with:

- decision nodes,
- player function assigning who acts,
- actions on edges,
- chance nodes with probabilities,
- terminal utilities,
- information sets representing imperfect information.

Perfect information: every information set is singleton. Imperfect information: some player cannot distinguish nodes in an information set.

Perfect recall: players never forget their own previous actions or information.

Strategies

Pure strategy specifies an action at every information set.

Mixed strategy is a distribution over pure strategies.

Behavioral strategy independently randomizes at each information set. With perfect recall, Kuhn's theorem says mixed and behavioral strategies are realization-equivalent.

Nash and subgame-perfect equilibrium

Nash equilibrium applies to strategies in the whole game.

Subgame-perfect equilibrium (SPE) requires Nash equilibrium in every subgame. In perfect-information games, compute by backward induction.

SPE rules out noncredible threats.

For imperfect-information games, use refinements such as sequential equilibrium; however the exam outline asks mainly for Nash and subgame perfection.

Sequence form and LP

For two-player zero-sum extensive-form games with perfect recall, sequence form represents realization probabilities of action sequences. It is exponentially smaller than normal form in many games.

Constraints enforce flow:

- root sequence probability 1,
- probability of sequence equals sum of probabilities of child sequences at information sets.

Zero-sum EFG equilibrium can be solved by LP in sequence form.

Regret minimization

If each player minimizes external regret, empirical play converges to coarse correlated equilibrium. In two-player zero-sum games, regret minimization converges to Nash equilibrium in average strategy.

Counterfactual Regret Minimization (CFR) decomposes regret by information sets and is a standard algorithm for large imperfect-information games such as poker.

Core update idea:

- compute counterfactual values,
- accumulate positive regrets,
- use regret matching to choose future actions.

3. Coalitional Games

Definition

A transferable-utility coalitional game is

$$(N, v),$$

where N is set of players and characteristic function

$$v : 2^N \rightarrow \mathbb{R}$$

assigns value to every coalition. Usually $v(\emptyset) = 0$.

An allocation/payoff vector $x \in \mathbb{R}^N$ distributes value of grand coalition.

Efficiency:

$$\sum_{i \in N} x_i = v(N).$$

Representation can be explicit, by listing $v(S)$ for all coalitions, but that takes $O(2^n)$ space. Compact representations include weighted voting games, graph games, marginal-contribution networks, and logical rules.

Basic classes

- Additive: $v(S) = \sum_{i \in S} w_i$.
- Superadditive: disjoint coalitions benefit from merging:

$$v(S \cup T) \geq v(S) + v(T).$$

- Convex/supermodular:

$$v(S \cup T) + v(S \cap T) \geq v(S) + v(T).$$

- Simple voting game: $v(S) \in \{0, 1\}$, winning or losing coalition.

Core

The core is set of efficient allocations no coalition can block:

$$Core(v) = \left\{ x : \sum_{i \in N} x_i = v(N), \sum_{i \in S} x_i \geq v(S) \ \forall S \subseteq N \right\}.$$

If core is nonempty, allocation is stable. Core may be empty. Convex games have nonempty core.

Shapley value

Shapley value is fair allocation satisfying axioms:

- efficiency,
- symmetry,
- dummy player,
- additivity.

Formula:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} (v(S \cup \{i\}) - v(S)).$$

Interpretation: expected marginal contribution of player i when players join in a uniformly random order.

Naive computation is exponential because all coalitions are considered.

Simple explanation

The Shapley value asks: if players arrive in random order, how much value does this player add at the moment they arrive? Averaging that marginal contribution over all orders gives a fair-share interpretation.

Example

For two players with $v(\{1\}) = 1$, $v(\{2\}) = 0$, and $v(\{1, 2\}) = 4$, player 1's marginal contribution is 1 if arriving first and 4 if arriving second. Player 1's Shapley value is $(1 + 4)/2 = 2.5$; player 2 gets $(0 + 3)/2 = 1.5$.

Simple voting games

A simple voting game has winning coalitions. Player power can be measured by pivotality.

Shapley-Shubik index: probability that player is pivotal in a random ordering.

Banzhaf index: number of coalitions where player is critical:

$$\beta_i = |\{S \subseteq N \setminus \{i\} : v(S) = 0, v(S \cup \{i\}) = 1\}|.$$

Normalize by total critical counts if a probability-like index is desired.

4. Auctions and Mechanism Design

Auction mechanisms

An auction specifies:

- allocation rule: who gets item(s),
- payment rule: what winners/participants pay,
- bidding language.

Desirable properties:

- efficiency: item allocated to highest value bidder,
- incentive compatibility: truthful bidding is optimal,
- individual rationality: participation nonnegative utility,
- revenue,
- computational tractability.

Single-item auctions

Common formats:

- English ascending auction,
- Dutch descending auction,
- first-price sealed-bid auction,
- second-price sealed-bid auction (Vickrey).

How to bid

Private value v_i .

Second-price auction: truthful bidding $b_i = v_i$ is weakly dominant. Winner pays second-highest bid.

First-price auction: bidder shades bid below value. In symmetric risk-neutral independent private values with n bidders and uniform $[0, 1]$ values:

$$b(v) = \frac{n-1}{n}v.$$

English auction under private values is strategically similar to second-price. Dutch auction is strategically similar to first-price.

Exam tip

For single-item private-value auctions: Vickrey/second-price encourages truthful bidding, first-price encourages bid shading, English is strategically like second-price, and Dutch is strategically like first-price.

Revenue equivalence

Under standard assumptions:

- risk-neutral bidders,
- independent private values,
- symmetric bidders,
- same allocation rule,

- same expected utility for lowest type,
- all mechanisms yield same expected revenue.

Optimal single-item auction

Myerson auction maximizes expected revenue using virtual values:

$$\varphi_i(v) = v - \frac{1 - F_i(v)}{f_i(v)}.$$

Allocate to bidder with highest nonnegative virtual value, charge threshold payment. For i.i.d. regular distributions, this is like second-price auction with reserve price.

Combinatorial auctions

Multiple items; bidders value bundles:

$$v_i(S), \quad S \subseteq M.$$

Winner determination:

$$\max \sum_i v_i(S_i)$$

subject to allocated bundles being disjoint. This is NP-hard in general.

Representations:

- explicit bundle values,
- OR bids,
- XOR bids,
- bidding languages using logical combinations,
- compact valuation classes such as additive, unit-demand, submodular.

VCG mechanism can make truthful bidding dominant and efficient, but may have computational and revenue issues.

Example

With items A and B, bidder 1 values bundle {A,B} at 10, bidder 2 values {A} at 6, and bidder 3 values {B} at 6. The best allocation gives A to bidder 2 and B to bidder 3 for total 12, even though bidder 1 has the highest single bundle bid.

Exam Checklist

- Define NFGs, mixed strategies, dominated strategies, Nash equilibrium.
- Solve zero-sum games by LP and state minimax theorem.
- Explain correlated and Stackelberg equilibria.
- Define EFGs, information sets, perfect recall, mixed vs. behavioral strategies.
- Explain Nash vs. subgame-perfect equilibrium and backward induction.
- Explain LP/sequence-form and regret/CFR approaches for EFGs.
- Define coalitional game, core, Shapley value, Shapley-Shubik, Banzhaf.

- Compare single-item auction formats, bidding strategies, revenue equivalence, Myerson optimal auction.
- Explain combinatorial auctions and winner determination.

AI Specialization 6: UIR

Decision making, planning, and coordination of autonomous systems of one or more robots.

Sources used: official CTU CourseWare page for B(E)4M36UIR, which lists lectures/labs on robotic paradigms, grid and graph planning, exploration, incremental planning, multi-goal/data-collection planning, curvature-constrained data collection, PRM, RRT, and risk-aware planning; CTU state-exam topic list; standard robotics planning material.

UIR in Plain English

UIR is robot planning in the physical world. The key difference from abstract planning is geometry, sensing, uncertainty, motion constraints, and coordination between robots.

1. Name the robot problem.

Control, path, exploration, tasks, or sampling.

2. Name the space.

World, grid, graph, or configuration space.

3. Give the method.

A*, D*, frontier, allocation, PRM/RRT.

Robot architectures

How sensing, planning, and acting are organized.

- Hierarchical: sense-plan-act.
- Reactive: fast behavior from perception.
- Hybrid: deliberative planner plus reactive layer.

Path and motion planning

Move from start to goal without collision.

- Configuration space represents robot poses.
- Grid/graph planning uses A* and variants.
- D*/D* Lite replan when the map changes.

Exploration

Plan to gain information, not just reach a known goal.

- Frontier = boundary between known free and unknown space.
- Information gain and entropy score useful observations.
- Exploration balances travel cost and new knowledge.

Multi-robot and multi-goal

Assign tasks and order visits.

- Centralized methods optimize globally but scale poorly.
- Decentralized methods scale but may be less optimal.
- Robotic TSP adds geometry and motion constraints.

Sampling planners

Avoid explicit full-space search in continuous spaces.

- PRM builds a roadmap.
- RRT grows a tree toward random samples.
- RRT*/PRM* add asymptotic optimality.

How to answer

Always mention physical constraints.

- What is the state/configuration?
- What is known vs. unknown?
- What guarantee does the planner have?

Exam sentence: Motion planning usually works in configuration space, where each point represents a full robot pose and obstacles become forbidden regions.

Three traps

- Path planning ignores some motion constraints; motion planning includes geometry/kinematics.
- Probabilistic completeness does not mean guaranteed success in finite time.
- RRT is not asymptotically optimal; RRT* is.

Use this page to separate robotics planning topics by what is being planned.

1. Robotic Paradigms and Control Architectures

Robot, embodiment, sensors, actuators

A robot is an embodied agent acting in the physical world.

Core parts:

- **Sensors:** measure robot/world state.
- **Actuators/effectors:** cause motion or physical interaction.
- **Controller:** maps perception/internal state to actions.

Embodiment means the robot has a physical body subject to geometry, dynamics, energy, time, uncertainty, and interaction constraints.

Sensor types:

- **Proprioceptive:** internal state, e.g. encoders, IMU, battery, joint positions.
- **Exteroceptive:** external world, e.g. lidar, camera, sonar, bumper, GPS.

Actuators include DC motors, servos, pneumatic/hydraulic actuators, propellers, wheels, legs, grippers.

Hierarchical paradigm

Pipeline:

$$Sense \rightarrow Plan \rightarrow Act.$$

Features:

- explicit world model,
- deliberative planning,
- long-horizon reasoning.

Advantages:

- systematic,
- can optimize globally,
- good for predictable environments.

Drawbacks:

- slow reaction,
- brittle if model is incomplete,
- frame problem and closed-world assumptions.

Reactive paradigm

Direct coupling from perception to action, often behavior-based:

$$Sense \rightarrow Act.$$

Examples: obstacle avoidance, wall following, potential fields, subsumption architecture.

Advantages:

- fast,
- robust to local changes,

- simple behaviors can be composed.

Drawbacks:

- weak global planning,
- can get stuck in local minima,
- harder to guarantee task completion.

Hybrid paradigm

Combines deliberative and reactive layers:

- high-level planner selects goals/subgoals,
- executive monitors execution,
- reactive layer handles fast control and safety.

Typical architecture:

Planner $\rightarrow$ *Sequencer/Executive* $\rightarrow$ *Behaviors/Controllers*.

Advantages:

- combines global reasoning with fast response,
- practical standard for autonomous robots.

Drawbacks:

- integration complexity,
- consistency between layers,
- debugging and timing issues.

Simple explanation

Hierarchical control thinks first and acts later; reactive control acts immediately; hybrid control uses both. Most real robots are hybrid because they need long-term goals and fast safety reactions at the same time.

2. Path and Motion Planning

Path vs. motion planning

Path planning: find collision-free geometric path.

Motion planning: find feasible trajectory respecting kinematics/dynamics, usually time-parametrized.

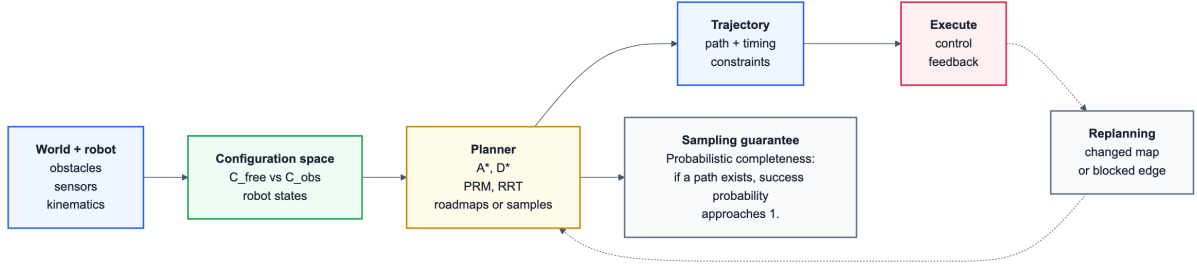


Figure 7: Robot planning pipeline: world and robot geometry are mapped to configuration space, planned over, executed, and replanned when the environment changes.

Configuration space

Configuration q fully describes robot pose/joint state. Configuration space:

$$\mathcal{C}$$

Obstacle region:

$$\mathcal{C}_{obs}$$

Free space:

$$\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{C}_{obs}.$$

Planning problem: find continuous path

$$\tau : [0, 1] \rightarrow \mathcal{C}_{free}$$

with $\tau(0) = q_{start}$ and $\tau(1) = q_{goal}$.

Simple explanation

Configuration space turns robot motion into point motion. Instead of reasoning about every point on a robot's body, we reason about one point in a higher-dimensional space where forbidden regions represent collisions.

Example

A circular mobile robot moving in a 2D map has configuration (x, y) if orientation does not matter. Obstacles in the workspace are inflated by the robot radius; then the robot can be treated as a point moving through the remaining free space.

Roadmap-based methods

A roadmap is a graph embedded in $\mathcal{C}_{free}$. Planning:

1. construct graph vertices/configurations,
2. connect nearby feasible configurations,
3. connect start and goal,
4. run graph search.

Examples:

- visibility graph,
- Voronoi diagram,
- probabilistic roadmap (PRM),
- lattice/grid roadmap.

Grid and graph planning

Discretize environment into cells or graph nodes. Use:

- BFS for unweighted graph,
- Dijkstra for nonnegative costs,
- A* with heuristic,
- Theta*/any-angle variants for smoother paths,
- weighted A* for speed at cost of optimality.

Speedups:

- better heuristics,
- bidirectional search,
- hierarchical planning,
- jump point search on grids,
- contraction/hub labels in road networks,
- incremental replanning.

Dynamic environments: D* and D* Lite

D* algorithms replan efficiently when edge costs change.

D* Lite is an incremental heuristic search algorithm based on Lifelong Planning A*. It maintains:

- $g(s)$: current best path cost,
- $rhs(s)$: one-step lookahead cost.

A state is locally consistent if $g(s) = rhs(s)$. When costs change, update affected vertices and repair shortest-path tree instead of replanning from scratch.

Useful for robots discovering obstacles while moving.

Exam tip

D* and D* Lite are not just “A* again.” Their point is reuse: when the map changes, repair the previous shortest-path computation instead of starting from zero.

3. Informative Path Planning and Exploration

Robotic exploration

Goal: build a map or gather information in initially unknown environment while navigating safely.

Common decomposition:

1. perception and mapping,
2. frontier/information target generation,
3. utility estimation,
4. task allocation among robots,
5. path planning to chosen target,
6. execution and replanning.

Frontier-based exploration

A frontier is boundary between known free space and unknown space. Algorithm:

1. maintain occupancy grid,
2. detect frontier cells,
3. cluster frontiers,
4. score each frontier by utility:

$$U = \text{information gain} - \lambda \cdot \text{travel cost},$$

5. navigate to best frontier.

Variants:

- nearest frontier,
- largest expected information gain,
- cost-utility frontier,
- multi-robot frontier assignment,
- hierarchical frontier selection.

Example

If frontier F1 is 5 meters away and reveals about 20 unknown cells, while F2 is 20 meters away and reveals 30 cells, a utility score like **gain - cost** may choose F1 because it gives more information per travel effort.

Information-theoretic methods

Use uncertainty measures:

- entropy:

$$H(X) = - \sum_x p(x) \log p(x),$$

- mutual information:

$$I(X; Z) = H(X) - H(X|Z).$$

Choose paths maximizing expected information gain:

$$\pi^* = \arg \max_{\pi} \mathbb{E}[I(\text{map}; \text{observations along } \pi)] - \text{cost}(\pi).$$

This is often computationally expensive, so approximations and sampling are used.

Simple explanation

Frontier exploration asks “where can I go to see new space?” Information-theoretic exploration asks “where can I go to reduce uncertainty the most?” The second is more principled but usually more expensive.

Multi-robot task allocation

Centralized methods:

- one planner assigns all robots,
- can optimize globally,
- communication/computation bottleneck,
- single point of failure.

Decentralized methods:

- each robot decides using local communication,
- scalable and robust,
- may be suboptimal.

Algorithms:

- greedy assignment,
- Hungarian algorithm for linear assignment,
- auctions/market-based allocation,
- consensus-based bundle algorithm,
- distributed frontier selection.

4. Multi-Goal and Data-Collection Planning

Multi-goal planning

Robot must visit multiple goals/targets. This is related to TSP.

If travel cost from target i to j depends on order, heading, or local path constraints, it becomes sequence-dependent TSP.

Robotic complications:

- obstacles,
- start and final depot,
- motion constraints,
- sensing radius,
- profits/rewards,
- time/energy budget,
- multiple robots.

TSP variants in robotics

- TSP: visit all targets at minimum cost.
- ATSP: asymmetric costs.
- GTSP: choose one target from each cluster.
- TSPN: visit neighborhoods of targets, not exact points.
- Dubins TSP: curvature-constrained vehicle.
- Orienteering problem: maximize collected reward within budget.
- Team orienteering: multiple robots.
- Prize-collecting TSP: trade travel cost against missed-target penalties.

Curvature-constrained trajectories

Dubins vehicle constraints:

- moves forward,
- bounded curvature,
- state (x, y, θ) .

Shortest path between configurations consists of circular arcs and straight segments, with types such as LSL, RSR, LSR, RSL, RLR, LRL.

For data collection with sensing radius, the robot may only need to pass near a target. This yields Dubins TSP with neighborhoods (DTSPN).

Routing with profits

Data collection often has more candidate measurements than can be visited.

Formulation:

$$\max \sum_i p_i y_i$$

subject to:

$$\text{travel_cost}(\text{route}) \leq B, \quad y_i = 1 \text{ iff target } i \text{ is visited/observed.}$$

This models limited battery/time and target rewards.

Solution methods

Decoupled methods:

1. choose sequence of targets,
2. compute geometric/motion-feasible path between consecutive targets.

Transformation methods:

- transform neighborhoods or headings into sampled configurations,
- reduce to TSP/ATSP/GTSP,
- solve with combinatorial solvers.

Sampling-based methods:

- sample viewpoints/configurations,
- connect feasible transitions,
- search for high-reward path in sampled graph.

Tradeoff: decoupled methods are simple but may be suboptimal; integrated sampling captures constraints but can be expensive.

5. Sampling-Based Motion Planning

Motivation

Exact motion planning in high-dimensional continuous configuration spaces is hard. Sampling-based planners avoid explicit construction of $\mathcal{C}_{free}$ and use collision checking.

Probabilistic Roadmap (PRM)

Multi-query planner:

1. sample configurations in $\mathcal{C}_{free}$,
2. connect nearby samples with local planner if collision-free,
3. store roadmap,
4. answer queries by connecting start/goal and searching roadmap.

Best for static environments with many queries.

Rapidly-exploring Random Tree (RRT)

Single-query planner:

1. start tree at q_{start} ,
2. sample random q_{rand} ,
3. find nearest tree node q_{near} ,
4. extend from q_{near} toward q_{rand} ,
5. add new collision-free node,
6. stop when goal reached.

RRT explores large spaces quickly and handles kinodynamic constraints with appropriate steering.

Example

Starting at q_{start} , RRT samples a random configuration. It finds the nearest existing tree node and extends a short step toward the sample. Repeating this makes the tree rapidly spread into unexplored regions of configuration space.

RRT*

RRT* improves RRT with rewiring:

1. choose best parent among nearby nodes,
2. rewire neighbors through new node if cost improves.

It is asymptotically optimal under assumptions.

Probabilistic completeness

Planner is probabilistically complete if, when a feasible path exists,

$$\Pr[\text{planner finds a path by time/samples } n] \rightarrow 1$$

as $n \rightarrow \infty$.

PRM and RRT are probabilistically complete under standard assumptions.

Asymptotic optimality

Planner is asymptotically optimal if solution cost converges to optimal cost:

$$\Pr \left[\lim_{n \rightarrow \infty} c_n = c^* \right] = 1.$$

RRT is generally not asymptotically optimal. RRT* and PRM* are.

Simple explanation

Probabilistic completeness means a planner will eventually find a path if one exists. Asymptotic optimality is stronger: with enough samples, the path quality converges to the best possible path.

Practical considerations

Important design choices:

- sampling distribution,
- nearest-neighbor data structure,
- collision checker,
- local planner,
- goal bias,
- step size,
- smoothing/post-processing,
- handling differential constraints.

Exam Checklist

- Compare hierarchical, reactive, and hybrid robotic paradigms.
- Define embodiment, sensors, actuators, proprioceptive/exteroceptive sensing.
- Define configuration space and free/obstacle regions.
- Explain roadmap, grid, graph planning, A*, and speedups.
- Explain D* / D* Lite for changing environments.
- Explain frontier exploration, information gain, and entropy.
- Compare centralized and decentralized multi-robot task allocation.
- Define robotic TSP variants, TSPN, Dubins TSPN, and routing with profits.
- Compare decoupled, transformation, and sampling-based multi-goal methods.
- Explain PRM, RRT, RRT*, probabilistic completeness, and asymptotic optimality.